GitHub Actions: Runners, Workflows & Real CI/CD

What is a Runner?

Definition:
A runner is a **machine (physical or virtual) that executes the steps of a GitHub Actions job**. It is where your build, test, and deployment commands actually run.

Key Characteristics:

- One job = one runner
- Each runner gets a **clean, isolated environment**
- Runners are **ephemeral** (temporary) for GitHub-hosted runners
- Runners can be **GitHub-managed** or **self-hosted**

Runners vs. Jenkins Agents (Mental Bridge)

Aspect	Jenkins	GitHub Actions
Agent/Runner	Agents (master connects to agents)	Runners (GitHub connects to runners)
Management	Install/manage yourself	GitHub manages (GitHub-hosted) or you manage (self-hosted)
Ephemeral	Agents persist across builds	GitHub-hosted runners are temporary (born → work → die)
Network Model	Agent pulls work from master	GitHub pushes work to runner
Isolation	Shared filesystem per agent	Fresh filesystem per job (GitHub-hosted)

Critical Insight:
In Jenkins, you worry about agent health, persistence, and cleanup. In GitHub Actions, you don't own the infrastructure; GitHub handles it or you opt into self-hosted control.

Types of Runners

1. GitHub-Hosted Runners

Definition:
Virtual machines managed entirely by GitHub. GitHub creates a fresh VM for each job, runs it, and destroys it after completion.

Available Options:

- ubuntu-latest (Linux, Ubuntu 22.04)
- ubuntu-20.04 (Linux, Ubuntu 20.04)
- windows-latest (Windows Server 2022)
- windows-2019 (Windows Server 2019)
- macos-latest (macOS, latest version)
- macos-12 (macOS 12 Monterey)
- macos-13 (macOS 13 Ventura)

Characteristics:

- ✅ No setup required
- ✅ Fresh environment every run (no stale dependencies)
- ✅ Pre-installed tools: git, Node, Python, Docker, Java, etc.
- ✅ GitHub handles maintenance and security patches
- ✅ Included free minutes per month (2,000 per month for public repos)
- ✅ Simple to use: `runs-on: ubuntu-latest`
- ❌ No persistent storage between runs
- ❌ No access to internal networks
- ❌ Limited customization

Best For:

- Public projects
- Standard build stacks (Node, Python, Java)
- Teams without compliance/network restrictions
- Cost-conscious teams (free tier available)

---

2. Self-Hosted Runners

Definition:
Your own machine (VM, bare metal, or cloud instance) that you register with GitHub and maintain. GitHub sends work to your runner; it executes and reports back.

Setup Concept:
Your Machine (On-Prem or Cloud)
↓
Install GitHub Actions Runner Software
↓
Register with GitHub repo/organization
↓
GitHub sends jobs to it
↓
Runner executes and sends logs back

Characteristics:

- ✅ Full control over environment and tools
- ✅ Access to internal networks and resources
- ✅ Persistent storage and caching
- ✅ No GitHub-managed minute limits
- ✅ Can customize OS, installed software, Docker images
- ✅ Cost-effective if you have existing infrastructure
- ❌ You manage installation, updates, security patches
- ❌ You ensure availability and monitoring
- ❌ You pay for the underlying infrastructure

Best For:

- Internal/private repositories
- Proprietary tools or custom stacks
- Compliance and security requirements
- Access to internal databases, APIs, VPNs
- Teams with DevOps expertise

Example Usage:
runs-on: [self-hosted, linux, x64]

Why Each Job Gets Its Own Runner (Benefits)

1. 🔒 Security – Isolation

Each job runs on a separate runner, preventing cross-job attacks or information leakage.

Job 1 (malicious/compromised code) → Runner A (isolated)
Job 2 (your real production deploy) → Runner B (isolated)

Job 1 cannot access secrets, files, or environment of Job 2.

For GitHub-hosted runners, the VM is destroyed after the job, making persistent compromise impossible.

2. 🧹 Clean State – No Interference

Each job starts with a fresh environment. No leftover files, cached artifacts, or configuration from previous jobs.

3. ⚡ Parallelism – True Concurrency

Multiple jobs run **simultaneously** on different runners, not sequentially on one shared agent.

Single shared runner:
Job1 (20 min) → Job2 (15 min) → Job3 (10 min) = 45 min total ❌

Multiple runners:
```
Job1 ──┐
Job2 ──┼─ All run in parallel = 20 min total ✅
Job3 ──┘
```

GitHub can spin up 20 runners in parallel; you're not queued.

4. 🐛 Debugging – Predictable Logs

Each job's logs show exactly what happened on **that** runner, uncontaminated by other jobs.

Job fails? Check ONLY that job's logs.
No need to ask: "Did Job 1 leave a bad cache?"
or: "Did Job 2 modify PATH?"

5. 📊 Performance – No Resource Contention

Each job gets full CPU, memory, and I/O of its runner. One job cannot starve another.

Shared runner:
Job 1 running heavy tests (8 CPU) → Job 2 waits, CPU throttled ❌

Separate runners:
Job 1: Full 8 CPU → Job 2: Full 8 CPU (different machine) ✅

Parallelism (No `needs:` directive)

```
jobs:
  lint:
    runs-on: ubuntu-latest
    steps: ...

  test:
    runs-on: ubuntu-latest
    steps: ...

  security-scan:
    runs-on: ubuntu-latest
    steps: ...
```

Without `needs:` directives, all three jobs run **in parallel** on three separate runners.

Sequential Execution (`needs:` directive)

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps: ...

  test:
    runs-on: ubuntu-latest
    needs: build # ← Wait for build
    steps: ...

  security-scan:
    runs-on: ubuntu-latest
    needs: test # ← Wait for test
    steps: ...
```

With `needs:`, jobs form a **pipeline**: build → test → deploy (sequential). Each waits for the previous.

Passing Data Between Jobs

Problem

Job 1 (on runner A) builds a file.
Job 2 (on runner B) needs that file.

How does it get there?

Solution: Use **artifacts** (for files) or **outputs** (for small values like version strings)

Environment Variables

Definition:
Variables that hold values (strings, paths, credentials) that steps can access.

Three Scopes

1. Workflow Level (All Jobs)

env:
NODE_VERSION: "18"

jobs:
test:
steps:
- run: echo $NODE_VERSION # ✅ Works

2. Job Level (Only That Job)

jobs:
test:
env:
TEST_ENV: "staging"
steps:
- run: echo $TEST_ENV # ✅ Works

3. Step Level (Only That Step)

jobs:
test:
steps:
- name: "Test"
env:
TEMP_VAR: "temporary"
run: echo $TEMP_VAR # ✅ Works

Secrets (Passwords, API Keys)

Definition:
Encrypted variables for sensitive data (passwords, tokens, API keys). Secrets are hidden in logs and only accessible to workflows.

Adding a Secret

Step 1: Go to **Settings → Secrets and variables → Actions**

Step 2: Click "New repository secret"

Step 3: Enter:
Name: DATABASE_PASSWORD
Value: your-secret-password

Using a Secret

- name: "Deploy"
  run: ./deploy.sh
  env:
  DB_PASS: ${{ secrets.DATABASE_PASSWORD }}

In logs, it appears as: `DB_PASS=***` (GitHub automatically masks secrets)

Secret Rules

- ✅ Use `${{ secrets.VARIABLE_NAME }}`
- ✅ GitHub automatically masks secrets in logs
- ✅ Create one secret per password
- ✅ Rotate secrets if someone leaves
- ✅ Use different secrets for different environments

Caching (Speeding Up Builds

Problem:
Downloading npm packages (500MB+) every run = slow.

Solution:
Cache the packages. First run: download and cache. Second run: reuse from cache (fast).

Automatic Caching (Recommended)

- uses: actions/setup-node@v4
  with:
  node-version: 18
  cache: 'npm' # ⚡ Magic: caches npm packages

Manual Caching (For Custom Paths)

- uses: actions/cache@v3
  with:
  path: ~/.npm
  key: *runner.os – npm –*{{ hashFiles('**/package-lock.json') }}
  restore-keys: |
  ${{ runner.os }}-npm-

How it works:

- `path:` = what folder to cache
- `key:` = unique identifier (includes hash of `package-lock.json`)
- If `package-lock.json` hasn't changed, cache is used
- If it changed, cache is invalidated, new packages downloaded

---

Interview-Ready Topics (Summary)

Question 1: "Explain GitHub Actions runner architecture."

Answer:

> "GitHub Actions has a control plane (managed by GitHub) that receives workflow definitions and schedules jobs to runners. Runners can be GitHub-hosted (managed by GitHub, ephemeral) or self-hosted (your own machines). Each job runs on a separate runner with an isolated filesystem. This provides security, clean state, and parallelism. Jobs can share data via artifacts (files) or job outputs (small values)."

Question 2: "Why doesn't each job share the same runner?"

Answer:

> "Separate runners for each job provide five benefits: (1) Security isolation – one compromised job can't access another's secrets. (2) Clean state – no leftover dependencies or configuration. (3) True parallelism – jobs run simultaneously, not sequentially. (4) Predictable debugging – logs show exactly what happened on that job. (5) No resource contention – each job gets full CPU/memory."

Question 3: "How do you pass data between jobs running on different runners?"

Answer:

> "Use artifacts for files and job outputs for small values. Upload artifacts with `actions/upload-artifact@v4` and download with `actions/download-artifact@v4`. For small values like version strings, use job outputs: define `outputs:` in the first job and access via `needs.job-name.outputs.variable-name` in the next job."

Question 4: "Explain the flow of a typical CI pipeline."

Answer:

> "A typical CI pipeline: (1) Event triggers workflow (e.g., push to main). (2) lint job starts on runner A – checks code style. (3) When lint finishes, test job starts on runner B – runs tests, uploads coverage artifact. (4) When test finishes, build job starts on runner C – builds application, uploads build artifact. Each job is independent; GitHub-hosted runners are ephemeral and destroyed after the job completes."

Question 5: "What are the 6 core components of GitHub Actions?"

> **Answer**:
>
> "Event → Workflow → Job → Step → Action → Runner. Event triggers workflow (YAML file). Workflow contains jobs. Each job runs on one runner. Job has sequential steps. Each step is either run: (shell command) or uses: (reusable action)."

Question 6: "How is GitHub Actions different from Jenkins architecture?"

> **Answer**:
>
> "Jenkins: You manage master (scheduler) + slaves (agents). GitHub Actions: GitHub manages control plane (hidden master). You only define jobs; GitHub schedules on runners (like agents). No server setup, YAML lives in repo."

Question 7: "How do you pass files between jobs on different runners?"

> **Answer:**
>
> "Job1: `actions/upload-artifact@v4` with name: 'build-output', path: dist/. Job2 (needs: job1): `actions/download-artifact@v4` with name: 'build-output'. Files appear in Job2 workspace for use."

Question 8: "What's the difference between `run:` and `uses:` in a step?"

> **Answer:**

"`run:` executes shell commands (npm test, echo). `uses:` calls reusable actions from marketplace (actions/checkout@v4, actions/setup-node@v4). Actions are pre-built, versioned, reusable logic."

Question 9: "3 jobs with `runs-on: ubuntu-latest`, no `needs:`. What happens?"

Answer:

"All 3 jobs start immediately in parallel on separate ubuntu-latest runners. No waiting. GitHub spins up 3 fresh VMs simultaneously."